Case Assignment 3

The ``Smarket'' dataset contains a 1250 days of S&P 500 index percentage returns. The data dictionary is as follows:

- Year: year of the return
- Lag1 - Lag5: 1 to 5 days lagged returns
- Volume: the number of shares traded on the previous day (in billions)
- Today: the percentage return on the day in question
- Direction: whether the return was positive or negative on the day in question

Create a training dataset of all days from 2004 and earlier. The test set is data from 2005.

1. Create a function to calculate the return on an investment of dollar value x over the entire dataset, invested before the first day in the dataset.
2. Create a function to calculate the return on an investment of dollar value x at the end of the day in question, if invested before the day in question began (i.e. before the percentage return was known).
3. Create and analyze a linear regression model to predict the percentage return.

4.Create and analyze a logistic regression model to predict whether the return will be positive or negative.ositive or negative. 5. Create a function to calculate the total return on an initial investment of dollar value x made using a few simple trading schemes: * at the beginning of any day, if you expect the percentage return to be positive, invest all of the money currently on hand, otherwise do not invest for the upcoming day. * at the beginning of any day, if you expect the percentage return to be greater than y%, invest all of the money currently on hand, otherwise do not invest for the upcoming day. * vary y and report your results. * how would you improve the above strategies? * how did you select the threshold for prediction with your logistic regression model? * how does threshold selection on the logistic regression model predictions differ from the selection of y? * remember that money currently on hand takes into account all past gains and losses as well as the initial investment * what is your best strategy?

In [106…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.formula.api import ols, logit
from sklearn.model_selection import train_test_split
import scipy.stats as ss
import seaborn as sns
from itertools import product


from ISLP import load_data
Smarket = load_data('Smarket')
#Smarket
```

In [107…
```
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

# Display the dataset
#print(Smarket)
```

In [108…
```
# split train and test data
```

In [109…
```
train_data = Smarket[Smarket['Year'] < 2005]
test_data = Smarket[Smarket['Year'] == 2005]

# Display the number of rows in each dataset
print("Training data shape:", train_data.shape)
print("Test data shape:", test_data.shape)

# Optionally, display the first few rows of each dataset
print("\nTraining Data (2004 and earlier):\n", train_data.head())
print("\nTest Data (2005):\n", test_data.head())
```

```
Training data shape: (998, 9)
Test data shape: (252, 9)

Training Data (2004 and earlier):
    Year   Lag1   Lag2   Lag3   Lag4   Lag5  Volume  Today Direction
0  2001  0.381 -0.192 -2.624 -1.055  5.010  1.1913  0.959        Up
1  2001  0.959  0.381 -0.192 -2.624 -1.055  1.2965  1.032        Up
2  2001  1.032  0.959  0.381 -0.192 -2.624  1.4112 -0.623      Down
3  2001 -0.623  1.032  0.959  0.381 -0.192  1.2760  0.614        Up
4  2001  0.614 -0.623  1.032  0.959  0.381  1.2057  0.213        Up

Test Data (2005):
       Year   Lag1   Lag2   Lag3   Lag4   Lag5  Volume  Today Direction
998    2005 -0.134  0.008 -0.007  0.715 -0.431  0.7869 -0.812      Down
999    2005 -0.812 -0.134  0.008 -0.007  0.715  1.5108 -1.167      Down
1000   2005 -1.167 -0.812 -0.134  0.008 -0.007  1.7210 -0.363      Down
1001   2005 -0.363 -1.167 -0.812 -0.134  0.008  1.7389  0.351        Up
1002   2005  0.351 -0.363 -1.167 -0.812 -0.134  1.5691 -0.143      Down
```

In [110…
```
#QUESTION 1 - Create a function to calculate the return on an investment of doll

def calculate_total_investment_return(data, initial_investment_amount):

    # Convert daily percentage returns to numerical format
    daily_percentage_returns = data['Today'] / 100

    # Compute cumulative return factor: (1 + daily_return) for each day
    cumulative_return_factors = (1 + daily_percentage_returns).cumprod()

    # Calculate the final value of the investment based on the cumulative return
    final_investment_value = initial_investment_amount * cumulative_return_facto

    return final_investment_value

# Example usage:
initial_investment_amount = 1000  # Example: $1000

# Calculate the total return on the investment over the entire dataset
final_investment_value = calculate_total_investment_return(Smarket, initial_inve
```

```
print(f"The final value of an initial investment of ${initial_investment_amount}
```

```
The final value of an initial investment of $1000 is: $959.53
```

In [111…

```
#QUESTION 2 - Create a function to calculate the return on an investment of doll

def calculate_return_on_investment(data, investment_value, day_index):
    # Ensure the day_index is within the bounds of the dataset
    if day_index < 0 or day_index >= len(data):
        raise IndexError("Day index is out of range.")

    # Investment is made before the percentage return for that day is known
    # so we use the return for that day to compute the end value
    daily_return = data['Today'].iloc[day_index] / 100
    final_value = investment_value * (1 + daily_return)

    return final_value

# Example usage:
investment_value = 1000  # Initial investment amount

# Calculate the return for the first 10 days as an example
#for day_index in range(10):
day_index=10
day_index-=1
final_value = calculate_return_on_investment(Smarket, investment_value, day_inde
print(f"Day {day_index+1}: Initial Investment = ${investment_value}, Final Value
```

```
Day 10: Initial Investment = $1000, Final Value = $1002.87
```

In [112…

```
# QUESTION 3 - Create and analyze a linear regression model to predict the perce

import pandas as pd
from ISLP import load_data
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.api as sm

# Load the dataset
Smarket = load_data('Smarket')

# Filter the data
train_data = Smarket[Smarket['Year'] <= 2004]
test_data = Smarket[Smarket['Year'] == 2005]

# Features and target variable
features = ['Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume']
X_train = train_data[features]
y_train = train_data['Today']

# Add constant to the training features
X_train = sm.add_constant(X_train)

# Initialize and fit the OLS model using statsmodels
ols_model = sm.OLS(y_train, X_train).fit()

# Prepare the test data
X_test = test_data[features]  # Select features for the test set
y_test = test_data['Today']   # Actual values for comparison
```

```
# Add constant to the test features
X_test = sm.add_constant(X_test)

# Predict the 'Today' variable using the OLS model
y_pred = ols_model.predict(X_test)

# Print predicted values and compare with actual
print("Predicted values:", y_pred.values)
#print("Actual values:", y_test.values)

#summary
print(ols_model.summary())

# Calculate performance metrics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the performance metrics
print(f"Mean Squared Error: {mse:.2f}")
print(f"R-squared: {r2:.2f}")
```

```
Predicted values: [ 9.68737668e-03 -6.99824466e-03  3.90222115e-02  2.79839617e-0
2
  1.29177056e-02  4.12978348e-02  4.29475800e-02  2.41823001e-02
 -1.36499262e-02  2.22496576e-02 -1.49086683e-02  1.97722804e-04
 -1.28110215e-04  5.82644380e-02 -5.82723373e-04 -1.35161658e-02
  4.23820560e-02  2.43810796e-02  2.31346912e-02  1.89482420e-02
 -3.03414735e-02 -3.83446806e-02 -1.43062362e-02  8.63961739e-03
 -5.59561953e-02 -3.71288030e-02 -1.33512353e-02  2.49255300e-02
 -4.12141465e-02 -1.00063370e-02 -7.32804148e-03  2.33797881e-02
 -2.29634233e-02 -8.92766651e-03  4.59717432e-03  2.49735266e-02
  8.90831527e-03  1.46853585e-02 -2.22697851e-02  6.05653776e-02
 -3.11453329e-02 -3.71928559e-02 -3.18748502e-02  4.20954156e-03
 -3.62225297e-02  7.61376324e-03  2.35895502e-02 -2.80423287e-02
  1.18746251e-02  2.14645634e-02  5.40793653e-02  2.45136503e-02
  3.64936119e-02 -4.58995023e-03  5.05745827e-02  6.70452307e-02
  1.42702654e-02  1.39037592e-02  2.56783557e-02  6.39796368e-02
 -1.63390281e-02  1.59976987e-03  1.45895944e-02  3.12529704e-02
 -5.82627997e-02  1.29232899e-03  1.35294814e-02  1.39924500e-03
 -9.93290718e-03 -1.45680655e-02  9.18676379e-03  7.65834999e-02
  6.25126503e-02  9.19038673e-03  5.08882078e-02  8.08362755e-02
  4.41543892e-02 -8.93935641e-03 -2.84245107e-02  5.97654367e-02
 -6.67020343e-02  5.44012926e-02 -3.25578189e-02  2.09318174e-02
 -6.36812547e-03  2.08763274e-02 -4.95566828e-02 -1.23427195e-02
 -1.00724199e-02 -2.44424460e-02  1.65456862e-02  2.88209221e-02
  9.94586714e-03  3.18243375e-02 -2.85489496e-02  2.02307965e-02
 -5.95856827e-03 -4.54244065e-02 -3.77413117e-02 -4.22520009e-02
 -6.88243815e-03 -3.02814250e-03 -2.23416051e-02  1.71171986e-02
  1.50850730e-03 -3.15643295e-02  1.33361733e-02  2.50154731e-02
 -2.92410912e-02  6.18204544e-03  2.28375789e-02 -7.25885851e-04
  1.71747500e-03  2.34809614e-03 -2.17982509e-02  1.85174035e-03
 -1.37848931e-02 -1.19890488e-02 -2.92951705e-03 -9.79779678e-03
  1.27180474e-02  4.40722586e-02  2.57885638e-02 -5.99033710e-03
  4.58787285e-02  5.31341135e-02  2.76799334e-03 -4.94222458e-02
  2.40213362e-02  3.39573702e-02 -3.43857005e-02 -5.07453394e-02
  2.27602661e-02 -1.75733231e-02 -4.91029485e-02 -2.71976187e-02
  3.34447577e-03 -6.25802600e-03 -1.84600173e-02  1.47731334e-02
  1.63481180e-02 -2.03261438e-02 -9.60866928e-03  1.91917573e-02
 -2.91270599e-02  2.82450184e-02 -9.87151611e-04 -2.72951931e-02
 -1.94178286e-02  4.98368217e-02  2.28818471e-02 -6.76901128e-03
 -1.45839375e-03  4.07343803e-02  2.29184621e-02  1.69893141e-02
 -2.49464752e-02  3.30944391e-02 -8.40922283e-03  3.25625222e-02
 -1.91869154e-03  4.54140628e-02  7.05580089e-03  2.76389003e-02
  2.58106760e-03  1.06909120e-02  1.29802577e-02  3.37015218e-02
 -1.39895963e-02  1.85079781e-02 -1.60471867e-02 -1.12601548e-02
 -4.88358787e-02  6.21666715e-03 -2.71329083e-03 -4.99108414e-02
  1.57091243e-02  3.28599469e-02 -1.76141921e-02  1.01833161e-02
  4.32033231e-02  4.38910738e-02  3.67238392e-02 -1.28862323e-02
  3.17881961e-02  4.17597112e-02  4.04661213e-02 -8.35212273e-03
 -1.32561449e-02 -4.25013296e-03  7.72614673e-03  2.51580442e-02
  2.14688396e-02  4.15368850e-02  2.28014313e-02  7.08530293e-02
  8.27583094e-02  4.45907435e-02  9.69358048e-03  2.38359413e-02
  4.91740621e-03  4.99854614e-02 -9.77806586e-03  8.82089761e-05
  1.46028146e-02  7.21311110e-03 -5.21199416e-02  7.72653256e-02
  2.39476936e-02 -7.65222886e-02 -3.97383149e-03  3.03913888e-02
  2.57079669e-02 -7.27545311e-02 -2.62801695e-02  8.93256561e-03
 -2.96332295e-02 -7.78173345e-03 -9.29193062e-03 -1.76894674e-02
  1.46614937e-02  7.93129862e-03 -2.58581197e-02 -2.18535705e-02
 -2.23296908e-03  3.68450831e-03 -2.00265780e-02 -4.70217057e-02
 -3.69265645e-02 -9.78615428e-04 -3.38976948e-03  1.33018534e-02
 -8.61084095e-03  3.09547808e-02  1.55831239e-02  2.61162827e-02
```

```
 -2.92706967e-02  1.71218774e-02  1.17131570e-02 -3.15220658e-04
  1.83822042e-02 -3.02140193e-03 -5.28375279e-03  1.36057817e-02
  1.58508905e-03 -5.89497532e-04  1.21651734e-02  9.22372481e-03
  1.65543353e-02  2.07783890e-02 -4.20892995e-03 -7.95077681e-03]
```

OLS Regression Results

Dep. Variable:	Today	R-squared:	0.002
Model:	OLS	Adj. R-squared:	-0.004
Method:	Least Squares	F-statistic:	0.3626
Date:	Wed, 25 Sep 2024	Prob (F-statistic):	0.903
Time:	17:50:50	Log-Likelihood:	-1620.8
No. Observations:	998	AIC:	3256.
Df Residuals:	991	BIC:	3290.
Df Model:	6		
Covariance Type:	nonrobust		

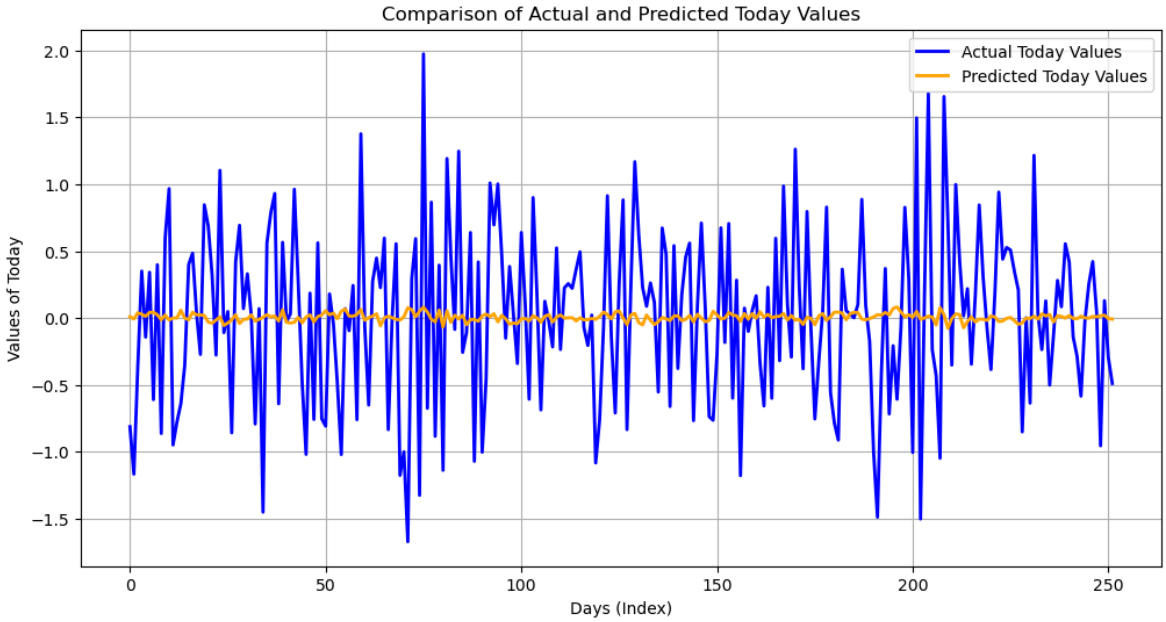
	coef	std err	t	P>|t|	[0.025	0.975]
const	-0.0146	0.205	-0.071	0.943	-0.417	0.388
Lag1	-0.0217	0.032	-0.684	0.494	-0.084	0.041
Lag2	-0.0106	0.032	-0.332	0.740	-0.073	0.052
Lag3	-0.0033	0.032	-0.104	0.917	-0.066	0.059
Lag4	-0.0060	0.032	-0.188	0.851	-0.068	0.056
Lag5	-0.0394	0.031	-1.254	0.210	-0.101	0.022
Volume	0.0110	0.147	0.075	0.940	-0.278	0.300

Omnibus:	49.299	Durbin-Watson:	2.000
Prob(Omnibus):	0.000	Jarque-Bera (JB):	143.022
Skew:	0.165	Prob(JB):	8.77e-32
Kurtosis:	4.825	Cond. No.	11.0

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Mean Squared Error: 0.42
R-squared: 0.00

In [113…

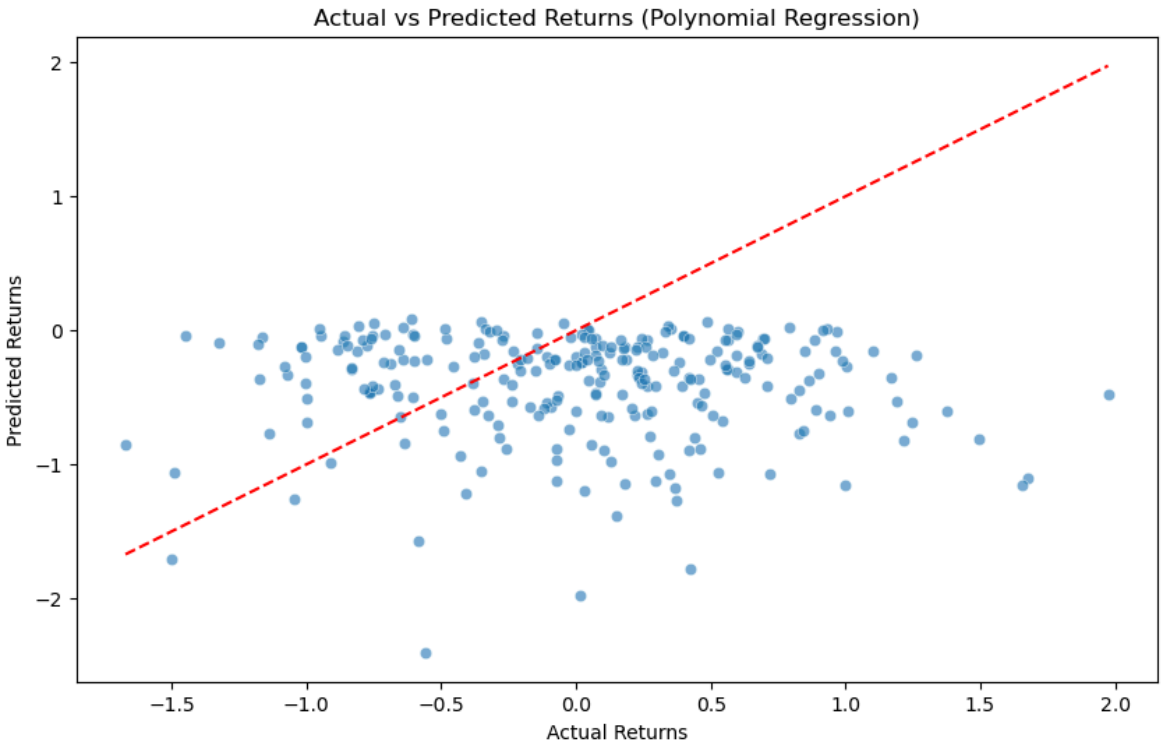
```
#Comparison of Actual and Predicted Today Values
plt.figure(figsize=(12, 6))
plt.plot(y_test.reset_index(drop=True), color='blue', label='Actual Today Values
plt.plot(y_pred.reset_index(drop=True), color='orange', label='Predicted Today V
plt.title('Comparison of Actual and Predicted Today Values')
plt.xlabel('Days (Index)')
plt.ylabel('Values of Today')
plt.legend()
plt.grid(True)
plt.show()
```



In [114…

```
#Polynomial Regression
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Feature Engineering: Add polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# Fit the polynomial regression model
model_poly = LinearRegression()
model_poly.fit(X_train_poly, y_train)

# Predict and evaluate
y_pred_poly = model_poly.predict(X_test_poly)
mse_poly = mean_squared_error(y_test, y_pred_poly)
r2_poly = r2_score(y_test, y_pred_poly)

print(f"Polynomial Mean Squared Error: {mse_poly:.2f}")
print(f"Polynomial R-squared: {r2_poly:.2f}")

# Plotting actual vs predicted values for polynomial regression
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred_poly, alpha=0.6)
plt.xlabel('Actual Returns')
plt.ylabel('Predicted Returns')
plt.title('Actual vs Predicted Returns (Polynomial Regression)')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', li
plt.show()
```

```
Polynomial Mean Squared Error: 0.77
Polynomial R-squared: -0.84
```



In [115…

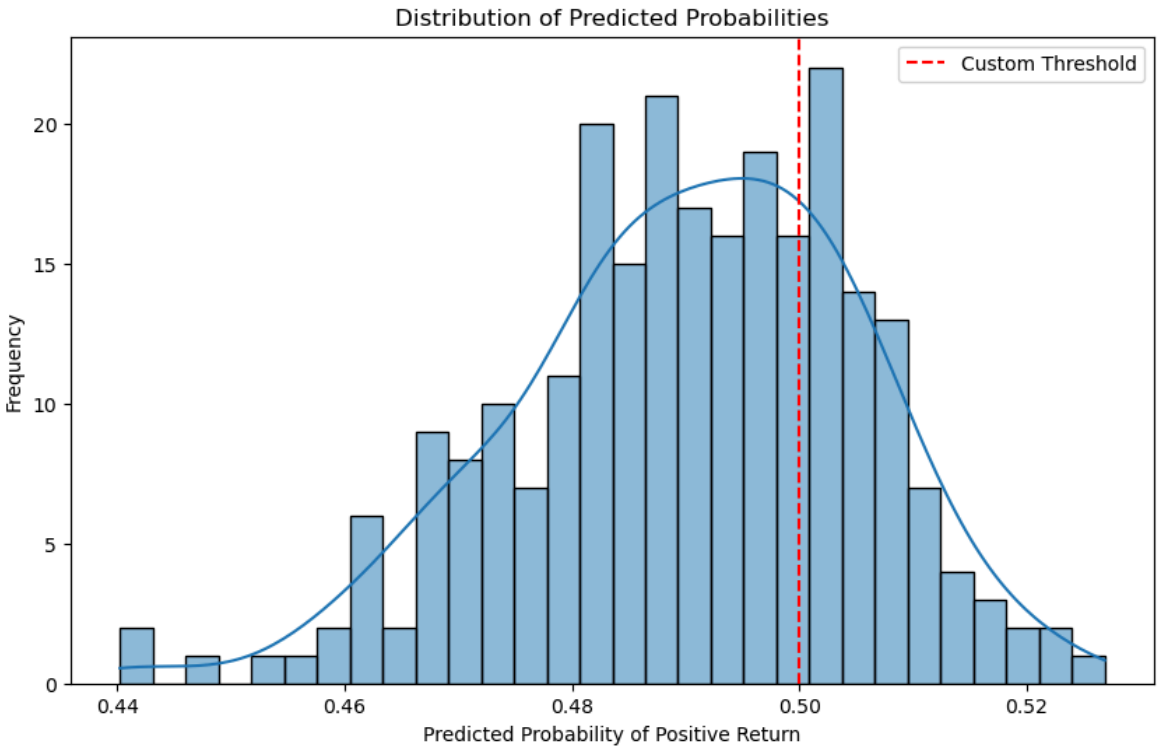
```
#QUESTION 4 - Create and analyze a logistic regression model to predict whether

import pandas as pd
from ISLP import load_data
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_sc
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
Smarket = load_data('Smarket')

# Create binary target variable: 1 if 'Today' > 0 (positive return), 0 otherwise
Smarket['PositiveReturn'] = (Smarket['Today'] > 0).astype(int)
# Filter the data
train_data = Smarket[Smarket['Year'] <= 2004]
test_data = Smarket[Smarket['Year'] == 2005]

# Features and target variable
features = ['Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume']
X_train = train_data[features]
y_train = train_data['PositiveReturn']
X_test = test_data[features]
y_test = test_data['PositiveReturn']

# Fit the Model
# Initialize and fit the logistic regression model
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)

# Predict probabilities on the test set
y_probs = logistic_model.predict_proba(X_test)[:, 1]  # Get probabilities for th

# Set a custom threshold
custom_threshold = 0.5
```

```
# Predict based on the custom threshold
y_pred_custom = (y_probs >= custom_threshold).astype(int)

# Performance metrics
accuracy = accuracy_score(y_test, y_pred_custom)
conf_matrix = confusion_matrix(y_test, y_pred_custom)
class_report = classification_report(y_test, y_pred_custom)

print(f"Accuracy: {accuracy:.2f}")
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(class_report)
```

```
Accuracy: 0.48
Confusion Matrix:
[[ 81  30]
 [101  40]]
Classification Report:
              precision    recall  f1-score   support

           0       0.45      0.73      0.55       111
           1       0.57      0.28      0.38       141

    accuracy                           0.48       252
   macro avg       0.51      0.51      0.47       252
weighted avg       0.52      0.48      0.46       252
```

In [116…

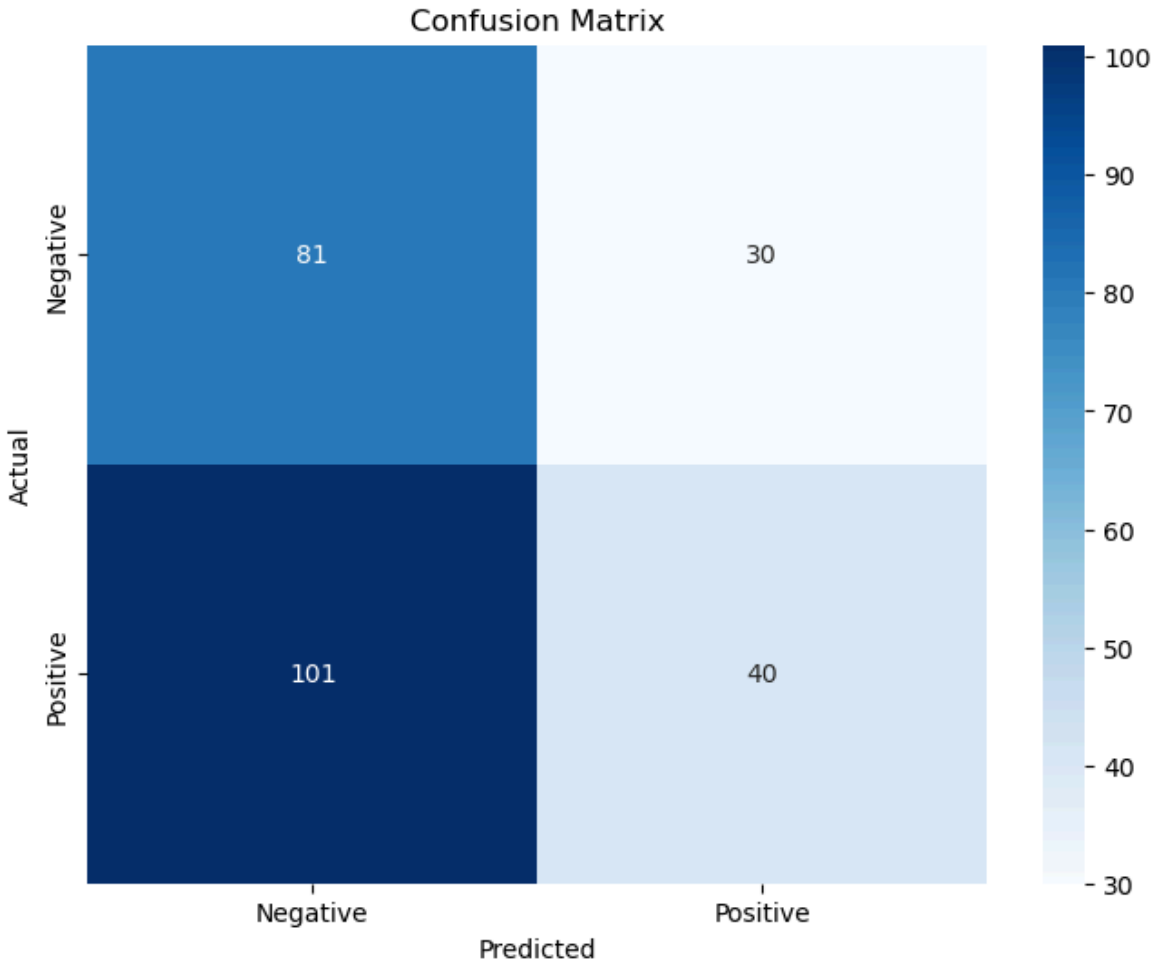
```
# Optional: Visualization of predicted probabilities
plt.figure(figsize=(10, 6))
sns.histplot(y_probs, bins=30, kde=True)
plt.axvline(x=custom_threshold, color='red', linestyle='--', label='Custom Thres
plt.xlabel('Predicted Probability of Positive Return')
plt.ylabel('Frequency')
plt.title('Distribution of Predicted Probabilities')
plt.legend()
plt.show()
```



In [117…

```
# Analyze Results
# Model coefficients
coefficients = pd.DataFrame(logistic_model.coef_[0], features, columns=['Coeffic
print("Model Coefficients:")
print(coefficients)

# Plotting confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Negati
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

```
Model Coefficients:
        Coefficient
Lag1      -0.059062
Lag2      -0.042119
Lag3       0.008779
Lag4       0.000466
Lag5      -0.004212
Volume    -0.120205
```



In [118…

```
import statsmodels.api as sm

# Add a constant to the features matrix (for the intercept)
X_train_with_const = sm.add_constant(X_train)

# Fit the logistic regression model
logit_model = sm.Logit(y_train, X_train_with_const)
logit_result = logit_model.fit()

# Print the summary
print(logit_result.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.691898
         Iterations 4
                           Logit Regression Results
==============================================================================
Dep. Variable:         PositiveReturn   No. Observations:                  998
Model:                          Logit   Df Residuals:                      991
Method:                           MLE   Df Model:                            6
Date:                Wed, 25 Sep 2024   Pseudo R-squ.:                0.001660
Time:                        17:50:51   Log-Likelihood:                -690.51
converged:                       True   LL-Null:                       -691.66
Covariance Type:            nonrobust   LLR p-value:                    0.8905
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.2015      0.334      0.604      0.546      -0.453       0.856
Lag1          -0.0592      0.052     -1.142      0.254      -0.161       0.042
Lag2          -0.0423      0.052     -0.817      0.414      -0.144       0.059
Lag3           0.0087      0.052      0.168      0.866      -0.093       0.110
Lag4           0.0003      0.052      0.007      0.995      -0.101       0.102
Lag5          -0.0043      0.051     -0.084      0.933      -0.105       0.096
Volume        -0.1268      0.240     -0.529      0.597      -0.596       0.343
==============================================================================
```

5. Create a function to calculate the total return on an initial investment of dollar value x made using a few simple trading schemes:
   - at the beginning of any day, if you expect the percentage return to be positive, invest all of the money currently on hand, otherwise do not invest for the upcoming day.

In [119…

```
def trading_strategy(data, initial_investment=1000, day_return=0):
    cash = initial_investment  # Start with the initial investment
    for i, row in data.iterrows():
        # If expected return (Today) is greater than the threshold
        if row['Today'] > day_return:
            # Invest all money, multiplying by the return percentage
            cash *= (1 + row['Today'] / 100)
    return cash
final_cash_0_return = trading_strategy(Smarket, day_return=0)
print(f"Final cash with 0% day return: ${final_cash_0_return:.2f}")
```

```
Final cash with 0% day return: $174605.77
```

Q.At the beginning of any day, if you expect the percentage return to be greater than y%, invest all of the money currently on hand, otherwise do not invest for the upcoming day.

In [120…

```
def trading_strategy_varying_y(data, initial_investment=1000, y_value=0):
    # Execute the trading strategy with a single threshold value
    final_cash = trading_strategy(data, initial_investment, y_value)
    return y_value, final_cash

# Test with a specific threshold value for y:
y_value = 0.5  # Test threshold of +0.5%
y, final_cash = trading_strategy_varying_y(Smarket, initial_investment=1000, y_v
```

```
# Print the result for the single threshold
print(f"Final cash with {y}% threshold: ${final_cash:.2f}")
```

```
Final cash with 0.5% threshold: $89398.31
```

In [121…
```
# * vary y and report your results.
```

In [122…
```
def trading_strategy_varying_y(data, initial_investment=1000, y_values=[0]):
    results = []
    for y in y_values:
        final_cash = trading_strategy(data, initial_investment, y)
        results.append((y, final_cash))
    return results
```

In [123…
```
# Now let's try a range of thresholds for y:
y_values = [-2, -1, 0, 1, 2]  # Test thresholds of -2%, -1%, 0%, +1%, +2%
results = trading_strategy_varying_y(test_data, initial_investment=1000, y_value

# Print results for different thresholds
for y, final_cash in results:
    print(f"Final cash with {y}% threshold: ${final_cash:.2f}")
```

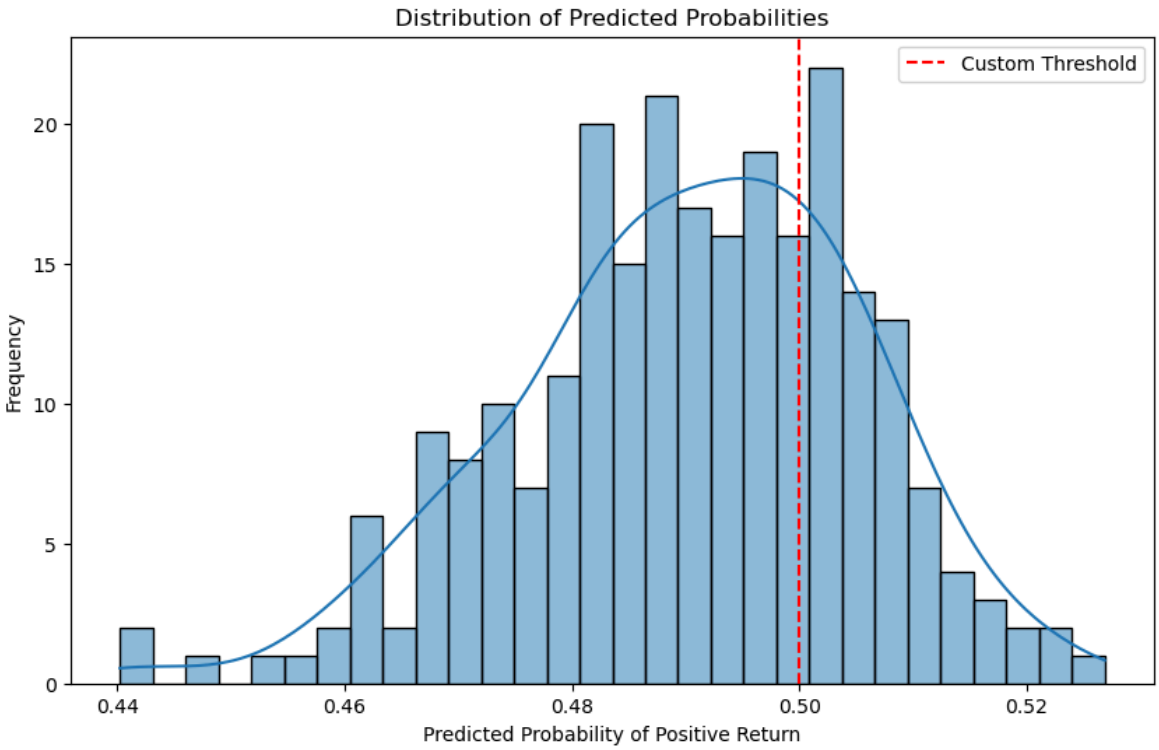
```
Final cash with -2% threshold: $1029.95
Final cash with -1% threshold: $1263.96
Final cash with 0% threshold: $1943.64
Final cash with 1% threshold: $1188.39
Final cash with 2% threshold: $1000.00
```

how would you improve the above strategies?

Adaptive Thresholds (Dynamic y) Volatility-Based Adjustments: Instead of using a static threshold, adjust y based on the market's volatility. For instance, in more volatile markets, a wider y range may be necessary, while in calm markets, a tighter y threshold would be appropriate. Volatility measures like the standard deviation or average true range (ATR) can guide this. Moving Average or Trend Indicators: Use moving averages or other trend-following indicators to dynamically adjust y. For example, if the market is in an uptrend (determined by the slope of the moving average), set a more aggressive y threshold, and if the market is flat or down, reduce the threshold.

In [124…
```
# how did you select the threshold for prediction with your logistic regression
```

In [125…
```
# Optional: Visualization of predicted probabilities
plt.figure(figsize=(10, 6))
sns.histplot(y_probs, bins=30, kde=True)
plt.axvline(x=custom_threshold, color='red', linestyle='--', label='Custom Thres
plt.xlabel('Predicted Probability of Positive Return')
plt.ylabel('Frequency')
plt.title('Distribution of Predicted Probabilities')
plt.legend()
plt.show()
```



answer : the threshold for prediction with your logistic regression model was done based using this graph

Q.how does threshold selection on the logistic regression model predictions differ from the selection of y?

The logistic regression threshold controls the probability required for the model to predict a positive return (e.g., a 50% probability threshold means the model must be at least 50% confident).

The y y threshold in the trading strategy controls the predicted return percentage needed to invest (e.g., only invest if the return is expected to be greater than 1%).

In summary:

Logistic regression threshold: Based on probability (confidence in positive return). y y threshold: Based on predicted return percentage (magnitude of return). They differ in what they measure: one is model confidence, the other is the size of the return.

Q.what is your best strategy? Dollar-cost averaging combined with stop-losses and diversification offers a solid, low-risk strategy for uncertain conditions. This allows you to participate in potential gains while managing the downside effectively

1. Dollar-Cost Averaging (DCA)

Description: Invest a fixed amount of money at regular intervals, regardless of market conditions. For example, you invest a fixed percentage of your portfolio each day. Rationale: This method averages out the purchase price over time. On bad days, you buy more at lower prices, and on good days, you buy less at higher prices. Benefit: It smoothens out market volatility and reduces the emotional burden of trying to time the

market. Best for: Long-term investors who want to mitigate risk without needing to predict market movements. 2.Set a Stop-Loss Limit Description: Place a stop-loss order that automatically sells part of your investment if the return drops below a certain percentage (e.g., -0.5%). Rationale: You may not know in advance if the day will be bad, but you can limit your losses once you realize it's a bad day. You can also use a trailing stop-loss, which adjusts as your investment grows. Benefit: Limits the downside during bad days while still allowing you to capture upside on good days.

In [ ]:

In [ ]:

In [ ]:

In [ ]: